

INFORME DE MATEMÁTICA Y SIMULACIÓN:

CÁLCULO DE PÉRDIDAS DE CARGA

Estudiantes:

Carlos Andrés Trujillo Ramírez - 20232217268
Jesus David Camacho García - 20222208357

Objetivo: Automatizar el cálculo iterativo de pérdidas de carga en tuberías y velocidad de flujo utilizando Scilab, documentando las funciones y estructuras basadas en la guía fundamental de programación con scilab.

Este es el código creado y utilizado en scilab para cumplir el objetivo

```
clc
// Datos de entrada (Cargados por defecto)
rugosidad = 0.0000015;
viscosidad = 0.000001007;
km = 11.8;
H_Total = 43.5;
diametro = 0.293;
longitud = 730;

// Parámetros del ciclo while
Tolerancia = 0.001;
maxIteraciones = 30;
error_hf = 1;
iteracion = 0;
H = H_Total;

// Creación del archivo de exportación (CSV)
fd = mopen('resultados_iteraciones.csv', 'wt');
fprintf(fd, "Iteracion,V,Hf\n");

// Ciclo Iterativo
while error_hf > Tolerancia & maxIteraciones > iteracion
    // Cálculo de velocidad y pérdidas
    velocidad = (-
2*sqrt(19.62*diametro*H)/sqrt(longitud))*(log10((rugosidad/(3.7*diametro))+((2.51*viscosidad*sqrt(lon
gitud))/(diametro*sqrt(19.62*diametro*H)))));
    perdidas_menores = km * (velocidad^2/(19.62));
    perdidas_por_friccion = H_Total - perdidas_menores;
```

```

// Cálculo del error con valor absoluto
error_hf = abs(perdidas_por_friccion - H);

// Guardar fila iterada en el archivo de texto
fprintf(fd, "%d,%.3f,%.3f\n", iteracion+1, velocidad, perdidas_por_friccion);

H = perdidas_por_friccion;
iteracion = iteracion + 1;
end

// Cierre del archivo exportado
fclose(fd);

// Pruebas Condicionales de Finalización
if iteracion >= maxIteraciones then
    disp("Se alcanzó el número máximo de iteraciones sin convergencia.");
else
    disp("Convergencia alcanzada en " + string(iteracion) + " iteraciones. Resultados exportados a resultados_iteraciones.csv.");
end

disp("La velocidad del fluido final es: " + string(velocidad));
disp("Las pérdidas por fricción finales son: " + string(perdidas_por_friccion));

```

Este código se guardó con extensión .sce para que cuando se ejecute aparte de realizar los cálculos y las iteraciones, guarde los resultados en un archivo .csv visualizable en Excel

1. Definición y Asignación de Variables

Según el estándar del software, las variables numéricas y matrices no requieren ser declaradas previamente; el sistema las reconoce dinámicamente al emplear el operador de asignación `=`. Las propiedades físicas del fluido (como la viscosidad `\mu`) y de la tubería (como la rugosidad `K_s` y el diámetro `d`) han sido preasignadas en el script para evitar pausas solicitando entradas manuales. Se usó el punto y coma (;) al final de cada asignación numérica para realizar el cálculo sin saturar la consola visualmente.

2. Estructuras de Control de Flujo (Ciclo Iterativo)

La convergencia de la ecuación se logró mediante un bucle **while ... end**, ideal para cuando se desconoce el número exacto de iteraciones requeridas para llegar a la solución. Este ciclo evalúa simultáneamente dos condiciones lógicas unidas por el operador **&**:

- Que el error actual (diferencia entre la pérdida iterada y la anterior) sea mayor a la tolerancia fijada de 0.001.
- Que el contador de iteraciones actual sea estrictamente menor al límite de seguridad (30), para evitar un bucle infinito en caso de divergencia matemática.

3. Funciones Analíticas de Scilab

Para la transcripción precisa de las ecuaciones hidráulicas mostradas en el problema base, se utilizaron varias funciones matemáticas nativas de Scilab:

- **sqrt(x)**: Computa la raíz cuadrada, aplicada frecuentemente en la fórmula de velocidad dependiente de la gravedad y el diámetro.
- **log10(x)**: Permite calcular el logaritmo en base 10 exigido por la ecuación logarítmica de fricción.

- **abs(x)**: Entregó el valor absoluto del margen de error (`error_hf`) para estandarizar la variable de control del ciclo **while**, evaluando siempre una distancia matemática positiva sin importar qué valor es superior.

4. Evaluación Condicional (Pruebas Lógicas)

Al concluir el ciclo repetitivo, la consistencia del cálculo se dictamina con un bloque **if ... else ... end**. El script verifica mediante operadores de comparación (`>=`) si el contador igualó el límite máximo. Esto garantiza una alerta transparente para el usuario sobre la validez de los resultados.

5. Visualización y Conversión de Datos

En el manejo de datos de salida, la función **disp()** es esencial. Para componer mensajes descriptivos que incluyan texto explicativo y variables numéricas cambiantes, el script emplea la función **string()**, que transforma la cifra (e.g. Iteración, Valor de H, Velocidad) en formato texto, y las une con el operador `+`.